Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas - AMS

Beatriz Cação Alamino
Enrico Meira De Paula
Felipe Salcedo Ramalho
Filipe Fogaça Lopes
Márcio Torres Rodrigues De Souza

Documentação de Desenvolvimento de Software

AMDE Digital - Controle de prontuário

Sorocaba
Dezembro - 2025

Beatriz Cação Alamino

Enrico Meira De Paula

Felipe Salcedo Ramalho

Filipe Fogaça Lopes

Márcio Torres Rodrigues De Souza

Documentação de Desenvolvimento de Software

AMDE Digital - Controle de prontuário

Projeto Integrador apresentado à Faculdade de Tecnologia de Sorocaba, como parte dos pré-requisitos para obtenção do título de Tecnólogo em Análise e Desenvolvimento de Sistemas do programa de Articulação do Ensino Médio e Superior do Centro Paula Souza

Orientador: Prof. Antonio Cesar de Barros Munari

Sorocaba

Dezembro - 2025

Dedicatória

Dedicamos este trabalho a todos os profissionais da área de tecnologia que continuam a inovar e a inspirar, também aos nossos familiares, que sempre nos apoiaram em todos os momentos, e aos professores da Fatec Sorocaba que nos auxiliaram e motivaram a concluir essa formação.

Agradecimentos

Agradecemos às nossas famílias por nos incentivarem a perseguir nossos sonhos e a manter a perseverança, estiveram ao nosso lado oferecendo apoio emocional e compreensão em todos os momentos dessa jornada.

Aos professores, que nos incentivaram e transmitiram valiosos conhecimentos, sempre nos mostrando diversos caminhos e uma visão crítica por meio de suas aulas, e, principalmente, das próprias experiências adquiridas ao longo de suas jornadas, especialmente ao professor e coordenador do nosso curso, Antonio Cesar Munari, por nos transmitir lições valiosas durante suas aulas, que nos motivou a não desistir e a encarar com divertimento os obstáculos que surgiram durante o desenvolvimento deste trabalho.

Aos colegas e amigos que, direta ou indiretamente, nos ajudaram com sugestões, palavras de apoio ou colaboração, deixamos aqui nosso sincero agradecimento.

Por fim, agradecemos a cada integrante deste grupo — Beatriz, Enrico, Felipe, Filipe e Marcio — pela dedicação, companheirismo e esforço coletivo que tornaram possível a realização deste trabalho.

Resumo

A Associação Amigos dos Deficientes (AMDE) é uma entidade voltada à prestação de serviços de saúde e reabilitação, que busca constantemente aprimorar seus processos de atendimento e gestão. Este Projeto Integrador apresenta o projeto AMDE Digital, um sistema integrado desenvolvido para modernizar a gestão de pacientes, prontuários e consultas clínicas, substituindo controles manuais e planilhas físicas por uma plataforma digital centralizada. O objetivo deste projeto é otimizar o atendimento, aumentar a eficiência operacional e garantir a segurança e acessibilidade dos dados clínicos, proporcionando uma experiência mais ágil tanto para profissionais quanto para pacientes. Para o desenvolvimento do sistema, foi realizado um levantamento de requisitos junto aos profissionais da instituição, identificando as principais necessidades, como autenticação segura de usuários, controle de permissões, gestão de prontuários digitais, upload de documentos, gestão de consultas e controle de presença. O AMDE Digital representa um avanço significativo na digitalização dos processos da clínica, reduzindo falhas humanas, agilizando o acesso às informações e reforçando a segurança dos prontuários médicos. Acredita-se que o sistema possa melhorar a qualidade do atendimento, otimizar a rotina dos profissionais de saúde e oferecer à instituição uma base tecnológica sólida para futuras expansões e inovações.

Palavras-chave: Gestão de Prontuários; Automação de Processos; Sistema Web e Mobile; Transformação Digital.

Lista de figuras

Figura 01 – Diagrama de Casos de Uso 11

Figura 02 – Modelo Conceitual 13

Figura 03 - Modelo lógico do banco de dados 13

Figura 04 – Diagrama de classe 14

Figura 05 – Diagrama de atividade da aplicação 14

Figura 06 – Tela de Login AMDE Digital 15

Figura 07 – Tela Inicial 15

Figura 08 – Tela de Pacientes 16

Figura 09 – Edição dos pacientes 16

Figura 10 – Tela de Usuários 16

Figura 11 – Tela de Agendamentos 17

Lista de Tabelas

Tabela 01. Custos estimados 11

Lista de Quadros

Quadro 1. Caso de uso – Autenticação de Usuário 11

Quadro 2. Caso de uso – Cadastro de Usuário 13

Quadro 3. Caso de uso – Consultar usuário 13

Quadro 4. Caso de uso – Atualizar Usuário 14

Quadro 5. Caso de uso – Excluir Usuário 14

Quadro 6. Caso de uso – Upload de Prontuário 15

Quadro 7. Caso de uso – Download de Prontuário 15

Quadro 8. Caso de uso – Excluir Prontuário 16

Quadro 9. Caso de uso – Criar Agendamento 16

Quadro 10. Caso de uso – Listar Agendamento 16

Quadro 11. Softwares necessários 17

Índice

1.Introdução 11

2. Referencial teórico 13

2.1 Prontuário Eletrônico do Paciente (PEP) 13

2.2 Gestão de Consultas e Controle de Presença 14

2.3 Segurança da Informação em Sistemas de Saúde 14

3. Análise de Requisitos 15

3.1. Visão geral do Produto 15

3.2 Descrição da Técnica Utilizada para o Levantamento dos Requisitos 16

3.3. Levantamento de Requisitos: 16

3.3.1 Requisitos Funcionais (RF) 16

3.3.2 Requisitos Não Funcionais (RNF) 17

3.4 Diagrama de Casos de Uso e Descrição dos Casos de Uso 17

4. Projeto Detalhado do Software 18

4.1 Arquitetura da aplicação 20

4.2. Tecnologias Utilizadas e APIs 21

4.3 Modelo de dados 22

4.3.1 Modelo Conceitual 22

4.3.2 Modelo Lógico 23

4.3.3 Diagrama de Classes 24

4.4 Diagrama de Atividades 24

4.5 Interfaces com o usuário 26

5. Implementação 26

5.1 Versionamento do Código – GitHub 27

5.2 Pipeline dos Projetos – GitHub Actions 29

5.3 Estágios de Commit – GitHub 29

5.4 Processo de Implantação e Entrega de Versão 30

5.5 Gerenciamento de Entrega Contínua (CI/CD) 37

5.6 Guia de Instalação do Sistema AMDE 37

5.6.1 Softwares Complementares Essenciais (Pré-requisitos) 38

5.7 Estimativa de Custos 38

6. Conclusão 39

Referências 39

Glossário 40

1.Introdução

A transformação digital na área da saúde vem se tornando um fator essencial para o aprimoramento dos serviços, otimização de processos e segurança das informações. No entanto, muitas instituições de pequeno e médio porte ainda enfrentam dificuldades em adotar soluções tecnológicas eficazes devido à falta de recursos, infraestrutura e conhecimento técnico especializado. A Associação Amigos dos Deficientes (AMDE) é um exemplo desse cenário: uma entidade comprometida com o atendimento e reabilitação de pessoas com deficiência, mas que ainda realiza a maior parte de sua gestão de forma manual, utilizando registros físicos e planilhas.

Essas práticas, embora comuns em instituições com recursos limitados, resultam em retrabalho, lentidão na recuperação de informações e maior risco de perda ou violação de dados sensíveis. Além disso, o gerenciamento de prontuários, consultas e presenças torna-se cada vez mais complexo à medida que a demanda cresce, exigindo uma ferramenta moderna e integrada que garanta eficiência e segurança nos processos internos.

Dentro desse contexto, este Projeto Integrador tem como objetivo o desenvolvimento do sistema AMDE Digital, uma plataforma centralizada e automatizada voltada à gestão de pacientes, prontuários e agendamentos clínicos. O sistema foi projetado para digitalizar e unificar os processos administrativos e operacionais da clínica, proporcionando maior agilidade no atendimento, controle de acesso seguro e melhor aproveitamento dos recursos humanos.

Para a concepção da aplicação, foi realizado um levantamento de requisitos junto aos profissionais da instituição, identificando as necessidades prioritárias, como autenticação de usuários, gestão de prontuários digitais, upload de documentos, controle de presença e agendamento automatizado de consultas.

Este documento tem como finalidade apresentar o processo de desenvolvimento do AMDE Digital, desde a concepção da ideia até a implementação final do sistema. O conteúdo está estruturado da seguinte forma:

Capítulo 1 descreve o contexto em que o projeto foi idealizado, bem como seus objetivos gerais e específicos. Capítulo 2 apresenta o referencial teórico, abordando temas relacionados à digitalização na saúde, gestão de prontuários eletrônicos e arquitetura de sistemas distribuídos. Capítulo 3 detalha o levantamento e análise dos requisitos funcionais e não funcionais identificados junto à instituição. Capítulo 4 explica o processo de desenvolvimento, as decisões de arquitetura, as tecnologias utilizadas e os modelos de

dados. Capítulo 5 apresenta os detalhes de DevOps e os detalhes de implementação, incluindo a estrutura dos módulos, repositórios e custos estimados no desenvolvimento do projeto. Por fim, o Capítulo 6 traz as conclusões do projeto, destacando os resultados alcançados e as perspectivas de evolução da solução.

2. Referencial teórico

Foi feito uma parceria para esse Projeto Integrador com a OSC AMDE, para isso é necessário definir uma Organizações da Sociedade Civil – OSC, elas são entidades sem fins lucrativos que objetivam cooperar com o Estado no atendimento ao interesse público, visando produzir transformações mediante a promoção de direitos sociais, conscientização socioambiental e combate à exclusão social, sobretudo no atendimento às pessoas em situação de vulnerabilidade (os mais frágeis da sociedade).

A transformação digital tem desempenhado um papel fundamental na modernização de diversos setores, incluindo o da saúde. A adoção de tecnologias da informação e comunicação (TICs) possibilita maior eficiência nos processos, redução de custos, integração de dados e melhoria na qualidade do atendimento. No entanto, a implementação de soluções tecnológicas em instituições de pequeno e médio porte ainda enfrentam desafios, como falta de infraestrutura, resistência à mudança e necessidade de capacitação profissional.

Segundo o Ministério da Saúde (2023), a informatização dos serviços de saúde é essencial para garantir a segurança e a integridade dos dados dos pacientes, promovendo uma gestão mais eficaz e transparente. Nesse contexto, sistemas de prontuário eletrônico do paciente (PEP) e gestão de agendamentos têm se destacado como ferramentas indispensáveis para clínicas, hospitais e instituições que buscam aprimorar seus fluxos de trabalho e melhorar a experiência de atendimento.

2.1 Prontuário Eletrônico do Paciente (PEP)

O Prontuário Eletrônico do Paciente é um sistema que reúne informações clínicas e administrativas em formato digital, substituindo os antigos registros em papel. Ele permite o armazenamento, a consulta e a atualização de dados médicos de forma rápida, segura e centralizada. De acordo com a Resolução CFM nº 1.821/2007, o PEP deve garantir autenticidade, integridade, confidencialidade e disponibilidade das informações, sendo um componente essencial para a continuidade do cuidado e para a tomada de decisões clínicas.

Além de contribuir para a redução de erros e a melhoria da comunicação entre profissionais de saúde, o PEP viabiliza uma gestão mais sustentável, eliminando o uso excessivo de papel e otimizando o espaço físico das instituições. No caso da AMDE, a

digitalização dos prontuários representa um avanço significativo, pois substitui os registros físicos armazenados em almoxarifado por um sistema de acesso controlado e auditável.

2.2 Gestão de Consultas e Controle de Presença

Outro aspecto relevante é o armazenamento dos dados consultas e controle de presença, que visa aprimorar a organização da rotina clínica. Sistemas informatizados permitem o gerenciamento eficiente das agendas dos profissionais, e o registro de comparecimento de pacientes e prestadores de serviço.

De acordo com estudos sobre gestão de processos em saúde (SILVA et al., 2021), o uso de ferramentas digitais para controle de agenda reduz faltas, melhora a comunicação interna e amplia a satisfação dos pacientes. Na AMDE, essa automação tem papel estratégico para otimizar o tempo de atendimento e assegurar a coordenação entre as diferentes áreas da instituição.

2.3 Segurança da Informação em Sistemas de Saúde

A segurança da informação é um dos pilares centrais de qualquer sistema que manipula dados sensíveis, especialmente no contexto da saúde. O armazenamento de informações médicas requer conformidade com normas de privacidade e proteção de dados, como a Lei Geral de Proteção de Dados Pessoais (LGPD – Lei nº 13.709/2018).

A aplicação de boas práticas de segurança, como o uso de criptografia de comunicação, e controle de acesso baseado em permissões, é essencial para garantir que apenas usuários autorizados tenham acesso a informações confidenciais. No projeto AMDE Digital, esses princípios são aplicados desde a concepção da arquitetura, reforçando a integridade e a confiabilidade do sistema.

3. Análise de Requisitos

3.1. Visão geral do Produto

A Associação Amigos dos Deficientes (AMDE) trata-se de uma entidade filantrópica e sem fins lucrativos que presta atendimento desde 2010 a crianças e adolescente com Transtorno Espectro Autista, através do Centro de Excelência em Autismo criou-se uma metodologia de trabalho, que engloba profissionais de diversas áreas, como serviço social, fonoaudiologia, terapia ocupacional, educação física, pedagogia, psicologia, fisioterapia, medicina psiquiátrica e um equipe de enfermagem.

O AMDE Digital é um sistema integrado voltado à gestão digital de pacientes, prontuários e consultas da Associação Amigos dos Deficientes (AMDE). Seu propósito é substituir processos manuais e registros físicos por uma plataforma centralizada, segura e acessível, capaz de otimizar o fluxo de informações e apoiar a tomada de decisões administrativas e médicas.

O sistema foi projetado para atender às necessidades da instituição por meio do seguinte módulo de acesso:

Portal Administrativo Web: destinado aos administradores e gestores, para o controle de cadastros de pacientes, prontuários, funcionários, agendamentos e relatórios.

O AMDE Digital incorpora mecanismos de autenticação e controle de acesso baseados em tokens JWT (JSON Web Tokens), assegurando que cada usuário acesse apenas as funcionalidades compatíveis com seu perfil — seja ele Administrador, Profissional de Saúde ou Colaborador de Apoio. O sistema também suporta upload de documentos e imagens, anexados aos prontuários dos pacientes, mantendo o histórico clínico completo e facilmente acessível.

Do ponto de vista operacional, o AMDE Digital proporciona melhor organização das agendas, redução do tempo de atendimento, maior confiabilidade das informações e segurança na manipulação dos dados sensíveis. A execução da aplicação e do banco de dados em containers Docker garante portabilidade, padronização e facilidade de implantação em diferentes ambientes.

Em resumo, o AMDE Digital representa uma solução tecnológica moderna e escalável, que visa digitalizar e integrar os processos internos da AMDE, promovendo eficiência, transparência e segurança na gestão das informações clínicas e administrativas da instituição.

3.2 Descrição da Técnica Utilizada para o Levantamento dos Requisitos

O levantamento de requisitos do sistema AMDE Digital foi realizado por meio de uma análise qualitativa e exploratória, combinando entrevistas semiestruturadas, observação direta e análise documental. Essas técnicas foram escolhidas por permitirem uma compreensão aprofundada das necessidades da instituição e dos desafios enfrentados em seu processo de gestão atual.

As entrevistas semiestruturadas foram aplicadas a colaboradores de diferentes setores da Associação Amigos dos Deficientes (AMDE) com o objetivo de identificar as principais dificuldades relacionadas ao registro de prontuários e controle de presença e consultas. Esse formato de entrevista possibilitou que os participantes expressassem suas percepções de maneira livre, ao mesmo tempo em que se mantinha o foco nos tópicos essenciais para o desenvolvimento do sistema.

A observação direta foi realizada durante o acompanhamento das rotinas diárias da instituição, permitindo analisar como os processos eram conduzidos manualmente, desde o cadastro de pacientes até o armazenamento e consulta dos prontuários físicos. Essa etapa foi fundamental para compreender as limitações dos métodos utilizados e identificar oportunidades de automação e integração.

Por fim, a análise documental envolveu o exame de planilhas, formulários e registros utilizados pela instituição, de modo a mapear o fluxo de informações e os tipos de dados manipulados. Essa verificação auxiliou na definição dos requisitos funcionais e não funcionais, assegurando que o sistema atendesse às reais demandas operacionais e administrativas.

A combinação dessas técnicas resultou em um conjunto de requisitos consistentes e alinhados às expectativas dos usuários, garantindo que o AMDE Digital fosse projetado de forma prática, segura e adaptada à realidade da instituição.

3.3. Levantamento de Requisitos:

3.3.1 Requisitos Funcionais (RF)

RF01 - Autenticação

O sistema deve permitir o login de funcionários e restringir o acesso a funcionalidades com base no nível de permissão do usuário (e.g., Administrador, Profissional).

RF02 - Gestão de Pacientes e Prontuários

O sistema deve permitir o cadastro, consulta, atualização e exclusão de informações de pacientes e seus respectivos prontuários digitais.

RF03 - Upload de Documentos

Deve ser possível anexar arquivos (PDFs, imagens) aos prontuários dos pacientes.

RF04 - Gestão de Agendamentos

O sistema deve permitir o cadastro de consultas e compromissos na agenda dos profissionais, validando a disponibilidade de horários.

RF05 - Controle de Consulta

O sistema deve permitir o registro de consultas realizadas envolvendo funcionários e pacientes nas atividades agendadas.

3.3.2 Requisitos Não Funcionais (RNF)

RNF01 - Segurança

A comunicação entre cliente e servidor deve ser segura. A autenticação das APIs será realizada via tokens JWT (JSON Web Tokens).

RNF02 - Plataforma

O back-end será desenvolvido em Java com Spring Boot. O portal administrativo será uma aplicação web (React).

RNF03 - Infraestrutura

A aplicação e o banco de dados (PostgreSQL) serão executados em containers Docker para portabilidade e escalabilidade.

RNF04 - Manutenabilidade

O código-fonte seguirá os padrões de mercado e a arquitetura será baseada em uma arquitetura monolítica para facilitar a manutenção e evolução independente dos módulos.

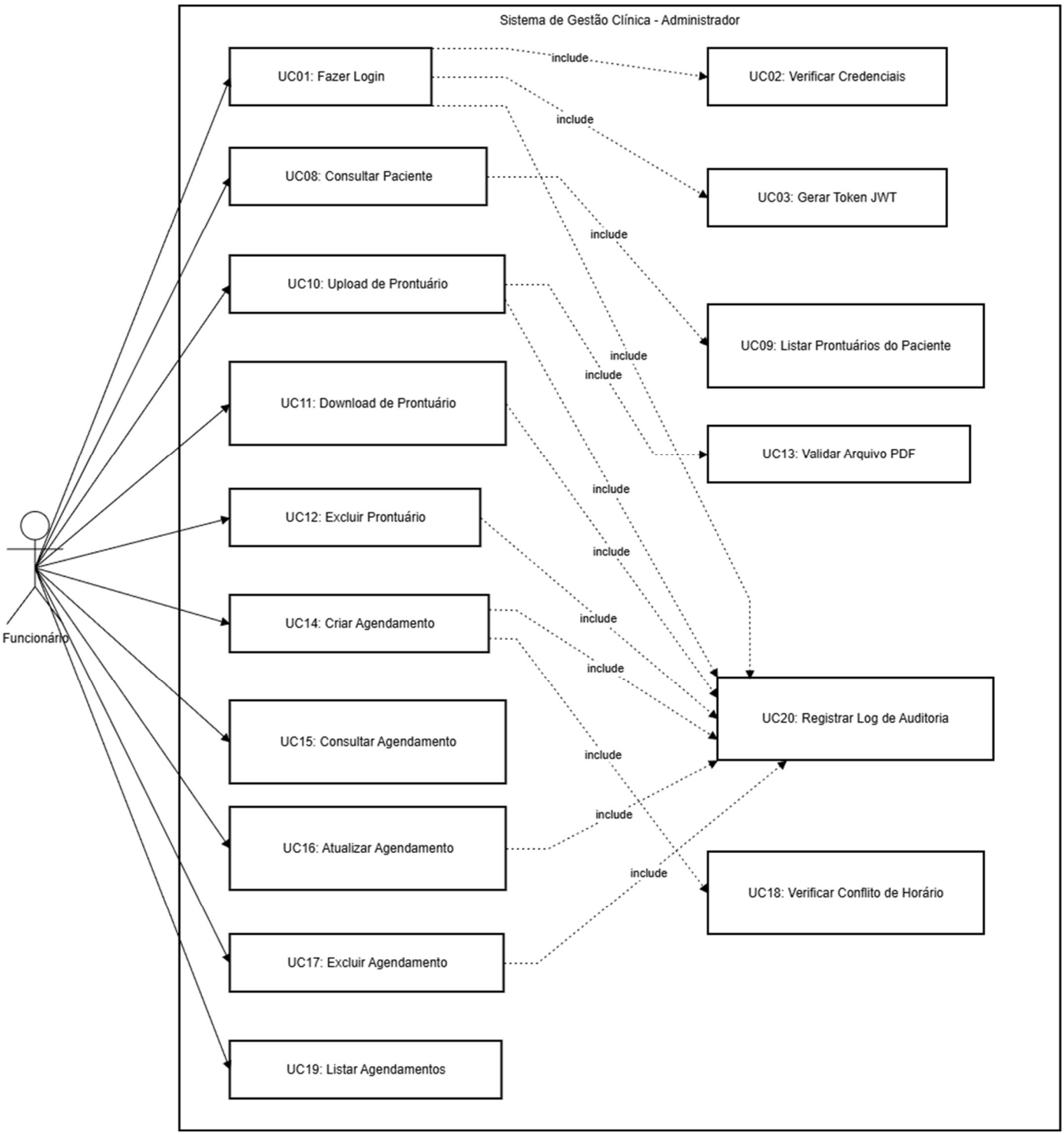
RNF05 - Controle de Acesso

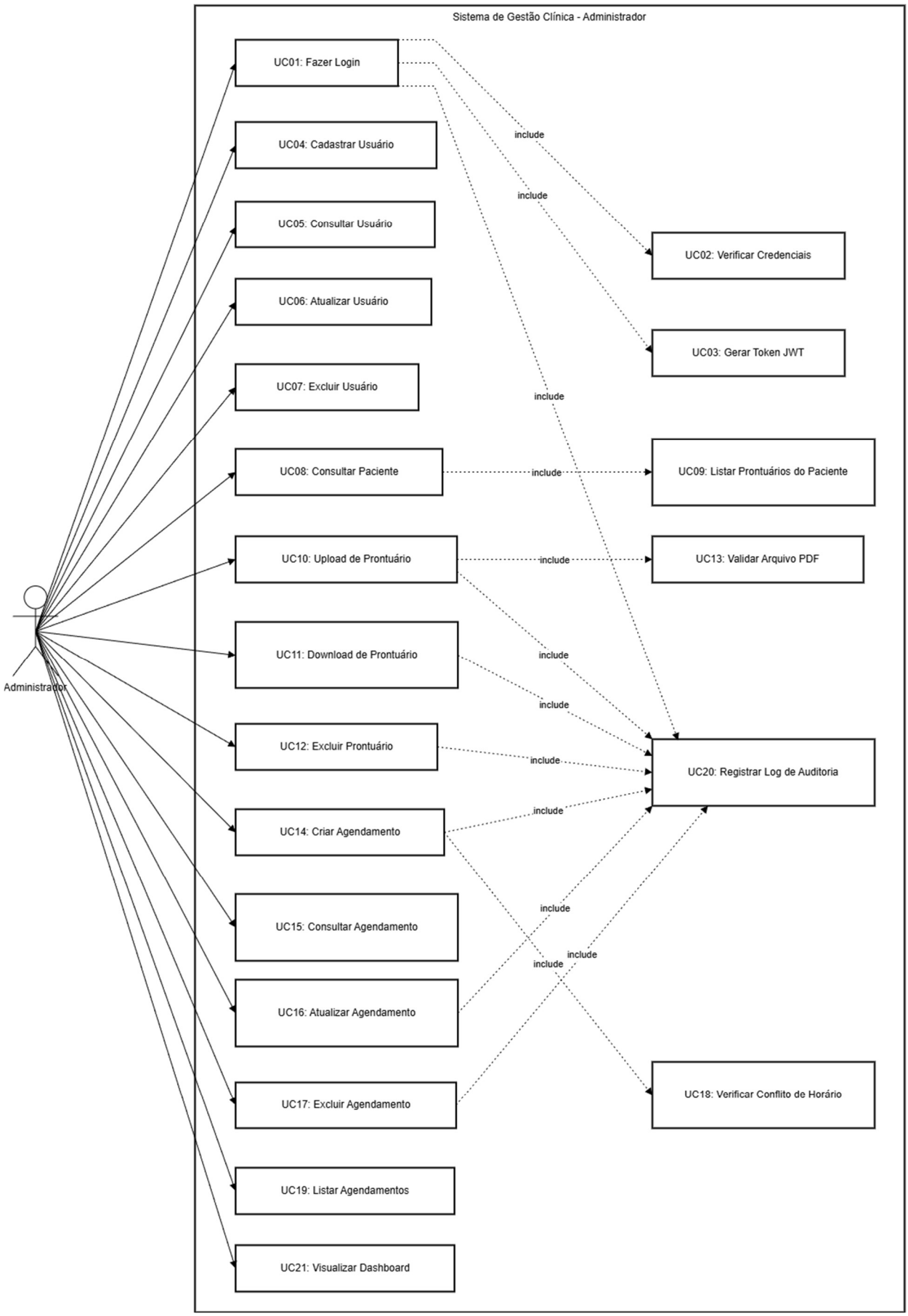
O sistema deve permitir o login de funcionários com credenciais seguras (e-mail e senha) e restringir o acesso a funcionalidades com base no nível de permissão do usuário (e.g., Administrador, Profissional).

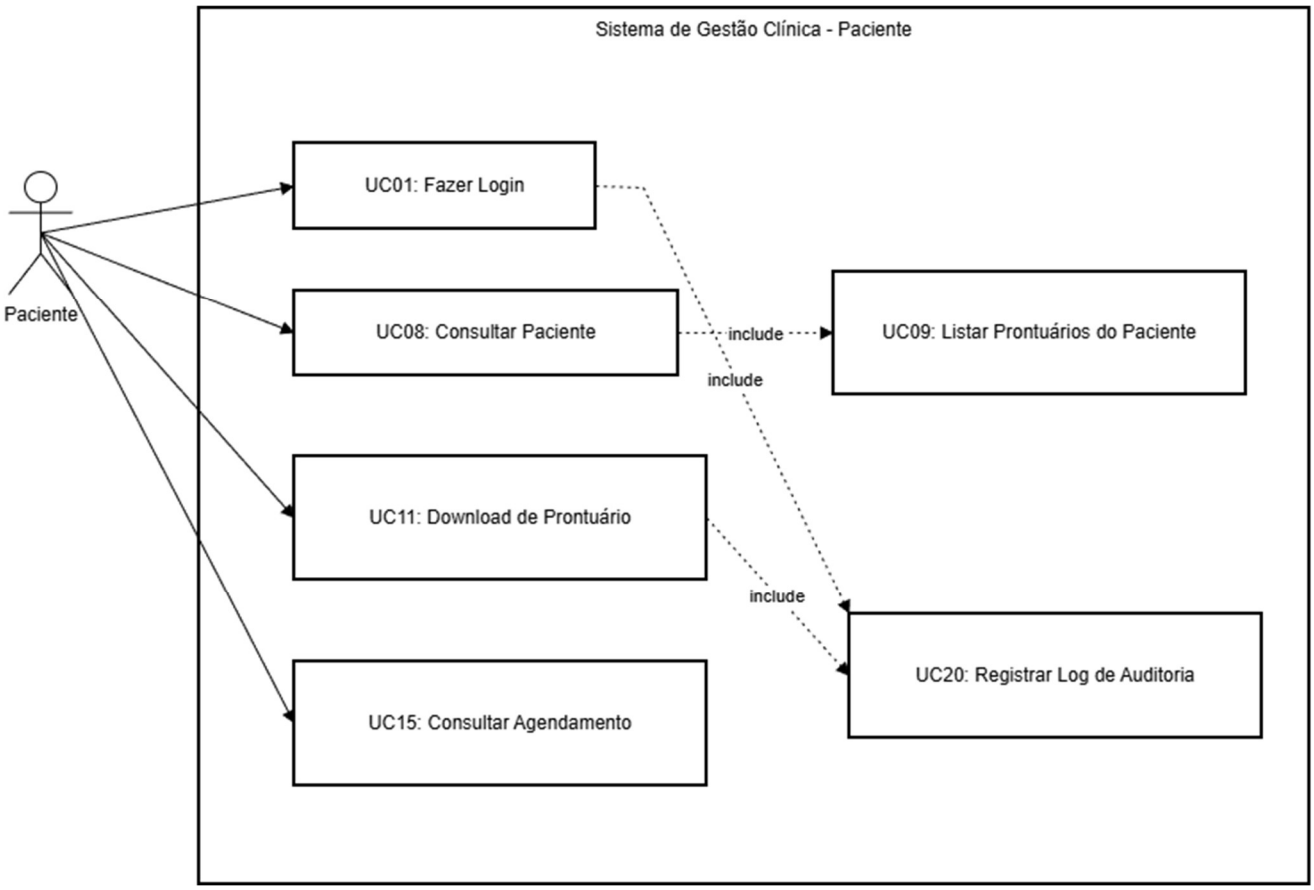
3.4 Diagrama de Casos de Uso e Descrição dos Casos de Uso

Na figura 01 é apresentado o diagrama de casos de uso do sistema.

Figura 01 – Diagrama de Casos de Uso




Sistema de Gestão Clínica - Administrador
Administrador
UC01: Fazer Login
UC04: Cadastrar Usuário
UC05: Consultar Usuário
UC06: Atualizar Usuário
UC07: Excluir Usuário
UC08: Consultar Paciente
UC10: Upload de Prontuário
UC11: Download de Prontuário
UC12: Excluir Prontuário
UC14: Criar Agendamento
UC15: Consultar Agendamento
UC16: Atualizar Agendamento
UC17: Excluir Agendamento
UC19: Listar Agendamentos
UC21: Visualizar Dashboard
UC02: Verificar Credenciais
UC03: Gerar Token JWT
UC09: Listar Prontuários do Paciente
UC13: Validar Arquivo PDF
UC20: Registrar Log de Auditoria
UC18: Verificar Conflito de Horário
include



Fonte: Autoria Própria

Quadro 1. Caso de uso – Autenticação de Usuário

Caso de Uso	UC01 - Fazer Login
Requisito Funcional	RF01 - Autenticação e Controle de Acesso
Atores	Paciente, Funcionário, Administrador
Pré-Condição	O usuário deve possuir cadastro prévio no sistema.
Pós-Condição	O sistema concede acesso com base no nivelAcesso e retorna token JWT.

Fluxo Principal	
Ações do Ator	**Ações do Sistema**

1 - Envia requisição POST /api/auth/login com email e senha.	
	2 - (Include) UC02 - Verifica credenciais (email e senha com BCrypt).
	3 - (Include) UC03 - Gera token JWT.
	4 - Retorna token de autenticação.
Fluxos de Exceção	
Condição	**Ação do Sistema**
2 - Credenciais incorretas.	Retorna erro 401 Unauthorized.

Quadro 2. Caso de uso – Cadastro de Usuário

Caso de Uso	UC04 - Cadastrar Usuário
Requisito Funcional	RF02 - Gestão de Usuários
Atores	Administrador
Pré-Condição	O Administrador deve estar autenticado.
Pós-Condição	O sistema cria novo usuário (Paciente ou Funcionário) com senha criptografada.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição POST /api/usuarios/registro com dados.	
	2 - Verifica se email já existe (usuarioRepository.findByEmail).
	3 - Criptografa senha com BCrypt.

	4 - Cria entidade Usuario, Paciente ou Funcionario.
	5 - Salva no banco de dados.
Fluxos de Exceção	
Condição	**Ação do Sistema**
2 - Email já cadastrado.	Retorna erro 409 Conflict.

Quadro 3. Caso de uso – Consultar usuário

Caso de Uso	UC05 - Consultar Usuário
Requisito Funcional	RF02 - Gestão de Usuários
Atores	Administrador
Pré-Condição	O Administrador deve estar autenticado.
Pós-Condição	O sistema retorna dados do usuário solicitado.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição GET /api/usuarios ou GET /api/usuarios/{id}.	
	2 - Busca usuário(s) no banco (usuarioRepository.findAll ou findById).
	3 - Converte para UsuarioDTO e retorna.

Quadro 4. Caso de uso – Atualizar Usuário

Caso de Uso	UC06 - Atualizar Usuário
Requisito Funcional	RF02 - Gestão de Usuários

Atores	Administrador
Pré-Condição	O usuário a ser atualizado deve existir.
Pós-Condição	O sistema atualiza os dados do usuário.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição PUT /api/usuarios/{id} com novos dados.	
	2 - Busca usuário (usuarioRepository.findById).
	3 - Atualiza campos modificados.
	4 - Salva alterações.

Quadro 5. Caso de uso – Excluir Usuário

Caso de Uso	UC07 - Excluir Usuário
Requisito Funcional	RF02 - Gestão de Usuários
Atores	Administrador
Pré-Condição	O usuário deve existir no sistema.
Pós-Condição	O sistema remove o usuário do banco de dados.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição DELETE /api/usuarios/{id}.	
	2 - Busca e remove usuário (usuarioRepository.delete).

Caso de Uso	UC10 - Upload de Prontuário
Requisito Funcional	RF04 - Gestão de Prontuários Digitais
Atores	Funcionário, Administrador
Pré-Condição	O paciente deve existir no sistema.
Pós-Condição	O sistema armazena o arquivo PDF e cria registro no banco.

Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição POST /api/prontuarios/upload com arquivo e dados.	
	2 - (Include) UC13 - Valida se arquivo é PDF e tamanho < 10MB.
	3 - Gera UUID para nome do arquivo.
	4 - Salva arquivo em uploads/prontuarios/paciente-{ID}/.
	5 - Cria RegistroProntuario e salva no banco.
	6 - (Include) UC20 - Registra log UPLOAD_PRONTUARIO.
Fluxos de Exceção	
Condição	**Ação do Sistema**
2 - Arquivo não é PDF ou > 10MB.	Retorna erro 400 Bad Request.

Quadro 6. Caso de uso – Upload de Prontuário

Quadro 7. Caso de uso – Download de Prontuário

Caso de Uso	UC11 - Download de Prontuário
Requisito Funcional	RF04 - Gestão de Prontuários Digitais
Atores	Paciente, Funcionário, Administrador
Pré-Condição	O prontuário deve existir no sistema.
Pós-Condição	O sistema retorna o arquivo PDF e registra log de auditoria.

Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição GET /api/prontuarios/{id}/download.	
	2 - Busca registro (registroProntuarioRepository.findById).
	3 - Carrega arquivo do storage (new UrlResource).
	4 - (Include) UC20 - Registra log DOWNLOAD_PRONTUARIO.
	5 - Retorna arquivo PDF.

Fluxos de Exceção	
Condição	**Ação do Sistema**
2 - Prontuário não encontrado.	Retorna erro 404 Not Found.
3 - Arquivo físico não existe.	Retorna erro 500 Internal Server Error.

Caso de Uso	UC12 - Excluir Prontuário
Requisito Funcional	RF04 - Gestão de Prontuários Digitais
Atores	Funcionário, Administrador

Pré-Condição	O prontuário deve existir no sistema.
Pós-Condição	O sistema remove o arquivo físico e o registro do banco.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição DELETE /api/prontuarios/{id}.	
	2 - Busca registro do prontuário.
	3 - Remove arquivo físico (Files.deleteIfExists).
	4 - Remove registro do banco.
	5 - (Include) UC20 - Registra log EXCLUSAO_PRONTUARIO.

Quadro 8. Caso de uso – Excluir Prontuário

Quadro 9. Caso de uso – Criar Agendamento

Caso de Uso	UC14 - Criar Agendamento
Requisito Funcional	RF05 - Gestão de Agendamentos
Atores	Funcionário, Administrador
Pré-Condição	Funcionário e Paciente devem existir no sistema.
Pós-Condição	O sistema cria agendamento com status "Agendado" e vínculo com paciente.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição POST /api/agendamentos com dados.	

	2 - Valida datas (dataInicio < dataFinal).
	3 - (Include) UC18 - Verifica conflito de horário do funcionário.
	4 - Cria Agendamento e AgendaPaciente.
	5 - Salva no banco.
	6 - (Include) UC20 - Registra log CRIACAO_AGENDAMENTO.
Fluxos de Exceção	
Condição	**Ação do Sistema**
2 - Datas inválidas.	Retorna erro 400 Bad Request.
3 - Conflito de horário.	Retorna erro 409 Conflict.

Quadro 10. Caso de uso – Listar Agendamento

Caso de Uso	UC19 - Listar Agendamentos
Requisito Funcional	RF05 - Gestão de Agendamentos
Atores	Paciente, Funcionário, Administrador
Pré-Condição	O usuário deve estar autenticado.
Pós-Condição	O sistema retorna lista de agendamentos conforme permissão do usuário.
Fluxo Principal	
Ações do Ator	**Ações do Sistema**
1 - Envia requisição GET /api/agendamentos.	
	2 - Identifica nivelAcesso do usuário.

	3 - Filtra agendamentos (Paciente: apenas seus / Funcionário: apenas seus / Admin: todos).
	4 - Retorna lista de AgendamentoDTO.

Fonte: Autoria Própria

4. Projeto Detalhado do Software

Este capítulo descreve a estrutura técnica do sistema, incluindo a arquitetura da aplicação, as tecnologias utilizadas, o modelo de dados e os diagramas de classes, atividades e fluxos. A modelagem foi elaborada com base nas boas práticas de engenharia de software e visa garantir escalabilidade, segurança, organização em camadas e facilidade de manutenção. Além disso, são apresentadas as interfaces da aplicação, projetadas com foco na experiência do usuário e nos princípios de usabilidade.

4.1 Arquitetura da aplicação

A arquitetura monolítica modular é um modelo de desenvolvimento de software em que toda a aplicação é construída como um único sistema integrado, porém organizada em módulos independentes internamente. Segundo Fowler (2015), essa abordagem combina a simplicidade de um monólito tradicional com a organização lógica dos módulos, permitindo uma melhor manutenção e evolução do código sem a complexidade de um sistema distribuído.

Para o AMDE Digital, a adoção de uma arquitetura monolítica modular foi uma escolha estratégica, pois garante simplicidade na implantação e manutenção, ao mesmo tempo em que mantém uma separação clara entre os módulos de pacientes, funcionários, agendamentos e prontuários. Essa estrutura facilita futuras expansões — como a adição de relatórios, notificações ou controle financeiro — sem necessidade de migrar para uma arquitetura mais complexa. Além disso, a execução em containers Docker assegura portabilidade e consistência entre os ambientes de desenvolvimento e produção.

O back-end da aplicação foi desenvolvido em Java utilizando o framework Spring Boot, que oferece suporte robusto para criação de serviços REST, segurança via JWT (JSON Web Tokens) e integração com o banco de dados PostgreSQL. O banco de dados é responsável pelo armazenamento de informações de pacientes, usuários, prontuários e registros de agendamentos, garantindo integridade referencial e consistência transacional.

A camada de front-end é composta por uma interface principal:

Portal Administrativo Web, desenvolvido em React, voltado aos administradores e gestores da clínica, com recursos de gerenciamento de cadastros, prontuários e relatórios;

Para assegurar portabilidade e padronização, tanto a aplicação quanto o banco de dados são executados em containers Docker, o que simplifica a implantação em diferentes ambientes e facilita a integração contínua (CI/CD). O servidor de autenticação é

responsável pela emissão e validação dos tokens JWT, garantindo o controle de acesso seguro conforme o perfil do usuário (Administrador, Profissional de Saúde ou Colaborador).

O sistema também adota boas práticas de segurança e desempenho, como criptografia de comunicações via HTTPS, controle de sessões, versionamento de APIs e respostas otimizadas com tempo médio inferior a 500ms em consultas padrão.

De forma geral, a arquitetura do AMDE Digital reflete uma solução distribuída, modular e segura, capaz de acompanhar o crescimento e as futuras demandas da instituição, garantindo confiabilidade, manutenibilidade e escalabilidade em todas as camadas da aplicação.

4.2. Tecnologias Utilizadas e APIs

A seleção das tecnologias utilizadas no desenvolvimento do sistema foi orientada pelos requisitos de desempenho, segurança e escalabilidade. O back-end foi desenvolvido em Java com o framework Spring Boot, que oferece suporte robusto a APIs REST e integração com bancos de dados relacionais, neste caso o PostgreSQL. O front-end foi planejado em React ou Angular para o portal administrativo web, e React Native ou Flutter para o aplicativo mobile dos profissionais.

A escolha dessas tecnologias assegura uma experiência moderna, responsiva e de fácil manutenção, alinhada às boas práticas do mercado de desenvolvimento de software.

Spring Boot (Java)

Spring Boot[1] é um framework open source baseado na linguagem Java, projetado para simplificar e acelerar a criação de aplicações stand-alone e de nível de produção, com o mínimo de configuração. Ele se baseia no ecossistema robusto do Spring Framework, mas remove o boilerplate (código repetitivo), permitindo que os desenvolvedores se concentrem na lógica de negócio. No projeto AMDE Digital, o Spring Boot é a espinha dorsal do back-end.

TypeScript (Front-end)

TypeScript[2] é um superset do JavaScript que adiciona tipagem estática opcional, ajudando a criar aplicações mais seguras, escaláveis e fáceis de manter. Ele é amplamente utilizado em projetos modernos de front-end e é a base de frameworks como React, Angular

[1] Disponível em: https://spring.io/projects/spring-boot.

[2] Disponível em: https://www.typescriptlang.org/

e Vue. No AMDE Digital, o TypeScript garante robustez e previsibilidade no código do front-end.

Prós:

Confiabilidade: Tipagem estática reduz erros em tempo de execução e melhora a qualidade do código.

Escalabilidade: Facilita o desenvolvimento de grandes aplicações com múltiplos módulos.

Produtividade: Sugestões inteligentes de IDE, autocompletar e melhor refatoração.

Contras:

Curva de Aprendizado: Exige entendimento de conceitos como generics, interfaces e tipos avançados.

Compilação: Requer um passo extra para transpilar para JavaScript, aumentando complexidade.

PostgreSQL

PostgreSQL[3] é um sistema gerenciador de banco de dados relacional (SGBDR) open source com foco em extensibilidade e conformidade com padrões SQL. Ele é frequentemente chamado de "o banco de dados relacional de código aberto mais avançado do mundo" devido à sua arquitetura robusta e recursos avançados.

No AMDE Digital, o PostgreSQL é o banco de dados principal, utilizado para garantir a persistência e integridade dos dados críticos, como prontuários de pacientes, dados de funcionários e registros de agendamento.

Prós:

Confiabilidade e Transações: Alta integridade de dados e conformidade rigorosa com ACID (Atomicidade, Consistência, Isolamento, Durabilidade).

Recursos Avançados: Suporte nativo a tipos de dados complexos (JSONB, Arrays, Geoespacial), tornando-o flexível.

Licença Permissiva: É totalmente *open source* e livre de custos de licenciamento.

Contras:

[3] Disponível em: https://www.postgresql.org/.

Complexidade de Configuração: Pode ser mais complexo de configurar para alta disponibilidade e escalabilidade massiva (em comparação com bancos NoSQL simples).

Docker

Docker[4] é uma plataforma que utiliza a virtualização no nível do sistema operacional (contêineres) para empacotar aplicações e todas as suas dependências em unidades portáteis e isoladas. Isso garante que o software funcione de maneira uniforme, independentemente do ambiente de execução.

Prós:

Padronização e Portabilidade: Elimina o problema "funciona na minha máquina", garantindo consistência do desenvolvimento ao ambiente de produção.

Isolamento de Processos: Os serviços operam em ambientes isolados, aumentando a segurança e a estabilidade.

Escalabilidade Simplificada: Facilita a implantação e o gerenciamento.

Contras:

Curva de Aprendizado: Exige o domínio de conceitos como `Dockerfile`, imagens e volumes.

Performance I/O: Em alguns sistemas operacionais, a performance de I/O (entrada/saída) de discos dentro de contêineres pode ser ligeiramente inferior.

React

React[5] é uma biblioteca JavaScript de código aberto, mantida pelo Facebook (Meta), utilizada para a construção de interfaces de usuário (UIs) focadas em componentização. Seu conceito central é o Virtual DOM (Document Object Model), que otimiza drasticamente a atualização e renderização dos elementos, garantindo alto desempenho e uma experiência fluida para o usuário.

No seu projeto (TCC), o React é a tecnologia Frontend principal, responsável por construir a interface web interativa e dinâmica que consome os dados e serviços fornecidos pelos microsserviços do backend (como os baseados em Spring Boot). Ele permite o

[4] Disponível em: https://www.docker.com/.

[5] Disponível em: https://react.dev/.

desenvolvimento de uma Single Page Application (SPA), onde a navegação ocorre sem recarregar a página inteira.

Prós:

Performance e Eficiência (Virtual DOM): Ao invés de manipular o DOM diretamente, o React compara o estado atual com o Virtual DOM anterior e atualiza apenas os componentes estritamente necessários, resultando em renderização muito mais rápida.

Componentização e Reutilização: A interface é dividida em componentes independentes e encapsulados. Isso facilita a manutenção, o teste e a reutilização de código (o mesmo botão ou bloco pode ser usado em várias telas).

Ecossistema Amplo e Maturidade: Possui uma comunidade enorme, ferramentas de desenvolvimento robustas (como o React DevTools) e a estabilidade de ser patrocinado por uma das maiores empresas de tecnologia.

Contras:

É Apenas a "View": React é uma biblioteca, não um *framework* completo. Para gerenciamento de estado global (como Redux ou Zustand) ou roteamento (como React Router), é necessário adicionar bibliotecas de terceiros, aumentando o número de dependências.

Rápido Ritmo de Mudanças: A comunidade e a biblioteca evoluem rapidamente. Manter-se atualizado com as novas *features* (como Hooks) e melhores práticas pode ser um desafio contínuo.

SEO Inicial (em SPAs): Por padrão, SPAs renderizam pouco conteúdo inicial, o que pode ser um problema para motores de busca. Exige soluções adicionais, como *Server-Side Rendering* (SSR) ou *Static Site Generation* (SSG), para otimizar o SEO.

Tailwind CSS

Tailwind CSS[6] é um framework CSS utilitário que permite estilizar componentes diretamente na marcação HTML usando classes pré-definidas. No AMDE Digital, ele acelera o desenvolvimento e garante um design padronizado e responsivo.

Prós:

Velocidade: Estilização direta no HTML reduz tempo de desenvolvimento.

Padronização: Facilita a criação de interfaces consistentes sem CSS customizado excessivo.

Performance: Gera CSS final extremamente enxuto.

Contras:

Legibilidade: Muitos utilitários no HTML podem deixar o código visualmente poluído.

Aprendizado inicial: É preciso memorizar grande quantidade de classes.

Shadcn/ui

Shadcn/ui[7] é uma biblioteca de componentes moderna que combina Tailwind CSS e Radix UI, fornecendo elementos acessíveis, personalizáveis e com design profissional. No AMDE Digital, ele agiliza a construção de interfaces complexas com aparência padronizada.

Prós:

Componentes Prontos: Reduz o tempo para construir elementos comuns (botões, cards, modais).

Altamente Personalizável: Código aberto e totalmente modificável.

Acessibilidade: Baseado em Radix UI, garantindo boas práticas de UX.

Contras:

Integração Manual: Os componentes são copiados para o projeto, exigindo manutenção.

Documentação Fragmentada: Parte no site, parte no GitHub.

Axios

Axios[8] é uma biblioteca JavaScript para realizar requisições HTTP de forma simples e eficiente.

[6] Disponível em: https://tailwindcss.com/

[7] Disponível em: https://ui.shadcn.com/

[8] Disponível em: https://axios-http.com/

Prós:

Facilidade de Uso: API simples, suporte a interceptors e cancelamento de requisições.

Compatibilidade: Funciona tanto no browser quanto no Node.js.

Gestão de Erros: Tratamento de erros mais intuitivo que o fetch nativo.

Contras:

Dependência Externa: Outra biblioteca para instalar e manter.

Tamanho: É maior que alternativas mais leves como fetch.

Spring Boot 3

Spring Boot 3[9] é a versão mais moderna do framework, trazendo suporte nativo ao Spring Framework 6, Java 17+ e Virtual Threads (Project Loom). No AMDE Digital, ele oferece estrutura robusta para APIs REST, segurança e integração com banco.

Prós:

Atualizado e Otimizado: Melhor performance, menor overhead.

Suporte a Virtual Threads: Melhora a escalabilidade de requisições.

Integração Completa: Funciona perfeitamente com Spring Cloud, Security, Data, etc.

Contras:

Compatibilidade: Exige versões mais novas do Java.

Complexidade: Mudanças em relação às versões anteriores requerem adaptação.

Spring Security com JWT

Spring Security[10] com tokens JWT oferece autenticação e autorização robustas, sem necessidade de manter sessões no servidor. No AMDE Digital, ele garante o acesso seguro às funcionalidades da plataforma.

Prós:

Escalabilidade: Tokens JWT dispensam sessões e funcionam bem em arquiteturas distribuídas.

Alto Nível de Segurança: Integração nativa com filtros, roles e políticas de acesso.

[9] Disponível em: https://docs.spring.io/spring-boot/

[10] Disponível em: https://docs.spring.io/spring-security/reference/index.html

Padronização: Muito utilizado em APIs REST modernas.

Contras:

Implementação Detalhada: Configurar filtros e providers exige atenção.

Revogação Difícil: JWTs não podem ser invalidados facilmente antes do vencimento.

Spring Data JPA

Spring Data JPA[11] é o módulo do Spring que abstrai a camada de persistência, simplificando operações com banco de dados usando JPA/Hibernate. No AMDE Digital, ele gerencia entidades, consultas e transações.

Prós:

Produtividade: Repositórios prontos reduzem muito o código.

Integração Total: Funciona perfeitamente com bancos relacionais.

Consultas Poderosas: Query Methods, JPQL e Criteria API.

Contras:

Abstração Pesada: Oculta complexidade do Hibernate, podendo gerar operações inesperadas.

Performance: Nem sempre ideal para cenários altamente performáticos.

Liquibase

Liquibase[12] é uma ferramenta de versionamento de banco de dados baseada em changelogs.No AMDE Digital, ele controla a evolução do esquema do banco de forma segura e automatizada.

Prós:

Segurança: Evita inconsistências entre ambientes (dev, homologação, produção).

Automação: Facilita CI/CD e rollback de alterações.

Flexibilidade: Suporte a XML, YAML, JSON e SQL.

Contras:

Complexidade Inicial: Exige aprendizado do conceito de changelogs.

Manutenção: Muitos arquivos podem dificultar organização.

[11] Disponível em: https://spring.io/projects/spring-data-jpa

[12] Disponível em: https://www.liquibase.com/

MapStruct

MapStruct[13] é um gerador de código que automatiza o mapeamento entre objetos Java (DTOs, Entities). No AMDE Digital, ele remove boilerplate ao transformar modelos.

Prós:

Performance: Gera código em tempo de compilação — nada de reflexão.

Produtividade: Reduz drasticamente código repetitivo.

Confiabilidade: Tipagem forte, evitando erros comuns de conversão.

Contras:

Configuração: Requer anotações e interfaces bem definidas.

Casos Complexos: Regras de mapeamento avançadas podem exigir customização manual.

Maven

Maven[14] é uma ferramenta de automação e gerenciamento de dependências para projetos Java. No AMDE Digital, ele organiza o build, controla dependências e integra com CI/CD.

Prós:

Padrão Consagrado: Estrutura clara (pom.xml) e repositórios estáveis.

Gerenciamento Automático: Resolve dependências e conflitos automaticamente.

Integração: Compatível com o ecossistema Spring e ferramentas de build.

Contras:

Verboso: Arquivos pom.xml podem ficar muito grandes.

Velocidade: Mais lento em comparação com Gradle em builds maiores.

4.3 Modelo de dados

4.3.1 Modelo Conceitual

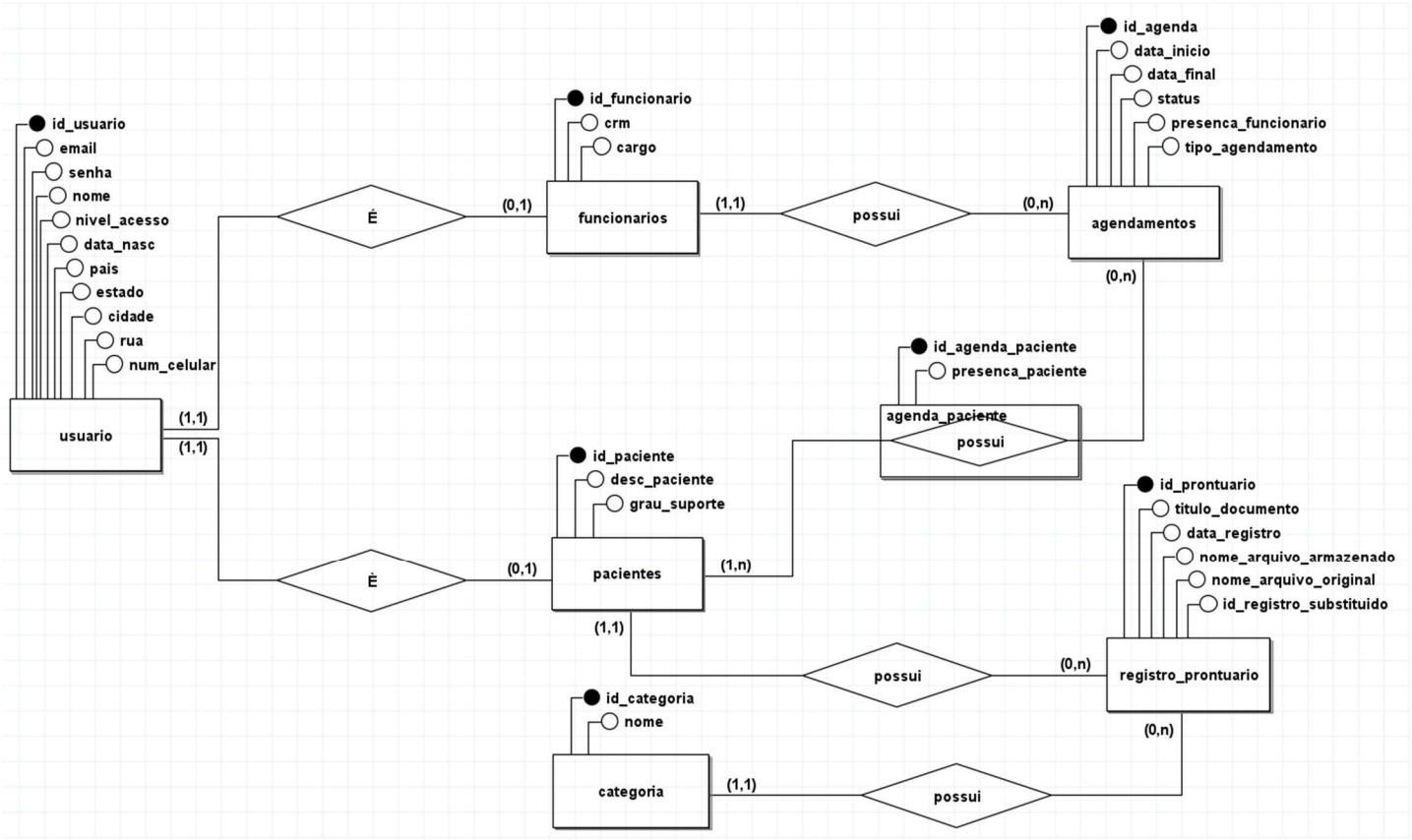
[13] Disponível em: https://mapstruct.org/

[14] Disponível em: https://maven.apache.org/

Para o desenvolvimento da plataforma AMDE Digital foi escolhido um banco de dados do tipo relacional, utilizado para garantir a persistência, integridade e consistência dos dados da aplicação. Esse modelo permite o uso de chaves primárias e estrangeiras, além de facilitar a implementação de consultas estruturadas em SQL.

A Figura 03 apresenta o Diagrama Entidade-Relacionamento (DER) utilizado na aplicação, no qual estão representadas as entidades principais (usuários, funcionários, pacientes, categorias, agendamentos e registros de prontuário), bem como seus respectivos atributos e relacionamentos.

Figura 02 – Modelo Conceitual

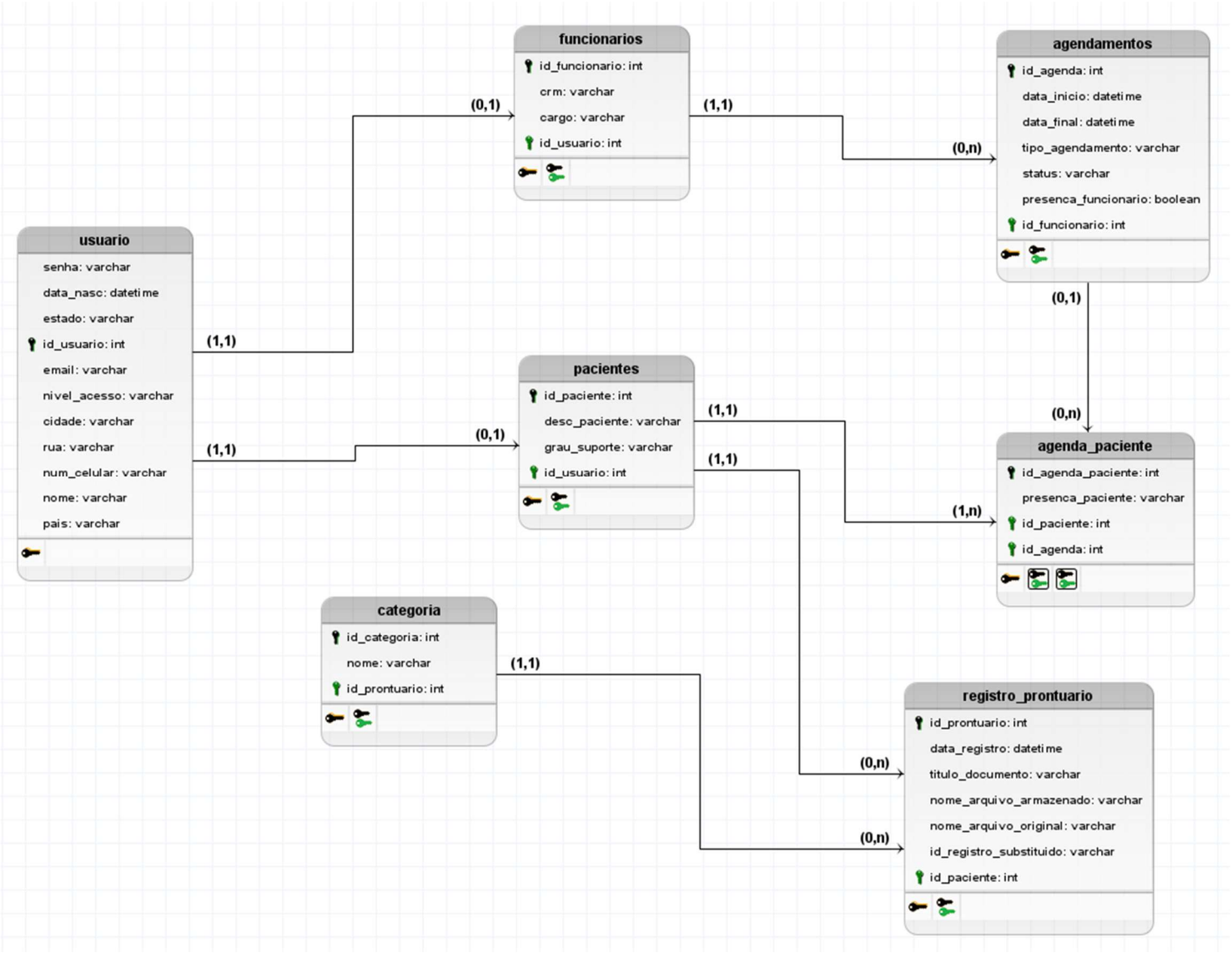


Fonte: Autoria Própria

4.3.2 Modelo Lógico

Na figura 04 é apresentado o modelo lógico do banco de dados.

Figura 03 - Modelo lógico do banco de dados

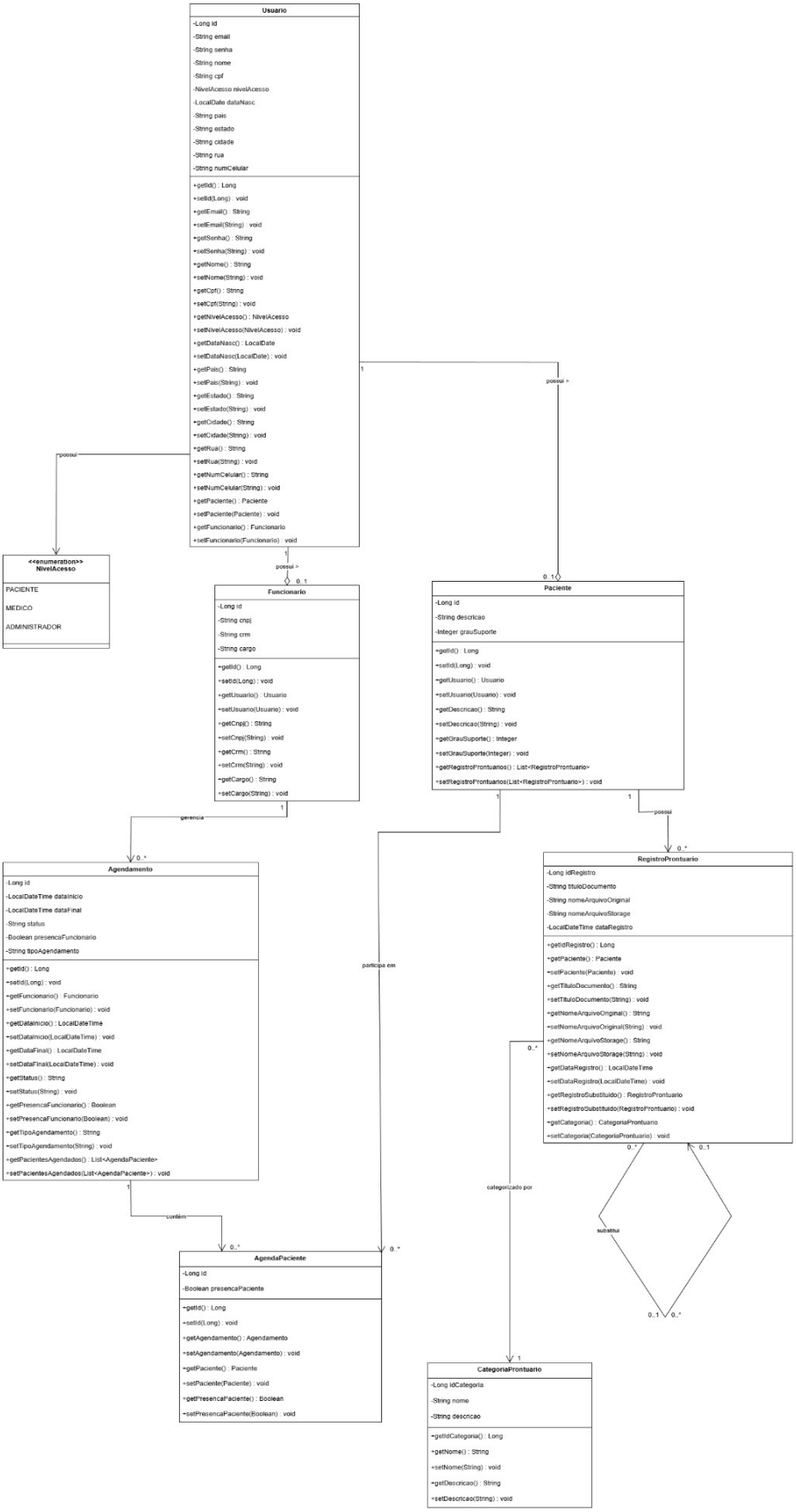


Fonte: Autoria Própria

4.3.3 Diagrama de Classes

Na figura 05 é apresentado diagrama de classe do projeto.

Figura 04 – Diagrama de classe



Fonte: Autoria Própria

4.4 Diagrama de Atividades

Um Diagrama de Atividades é uma representação visual utilizada para demonstrar a sequência de ações realizadas em um processo, incluindo os fluxos de controle e as principais decisões envolvidas.

Na Figura 06 é apresentado o Diagrama de Atividades da aplicação, contemplando três operações essenciais do módulo de usuários: login, listagem de usuários e registro de novos usuários. O diagrama ilustra, de forma clara, como as requisições enviadas pelo cliente percorrem as camadas da aplicação (Controller, Service e Repository), bem como as validações realizadas em cada etapa antes do retorno da resposta ao usuário.

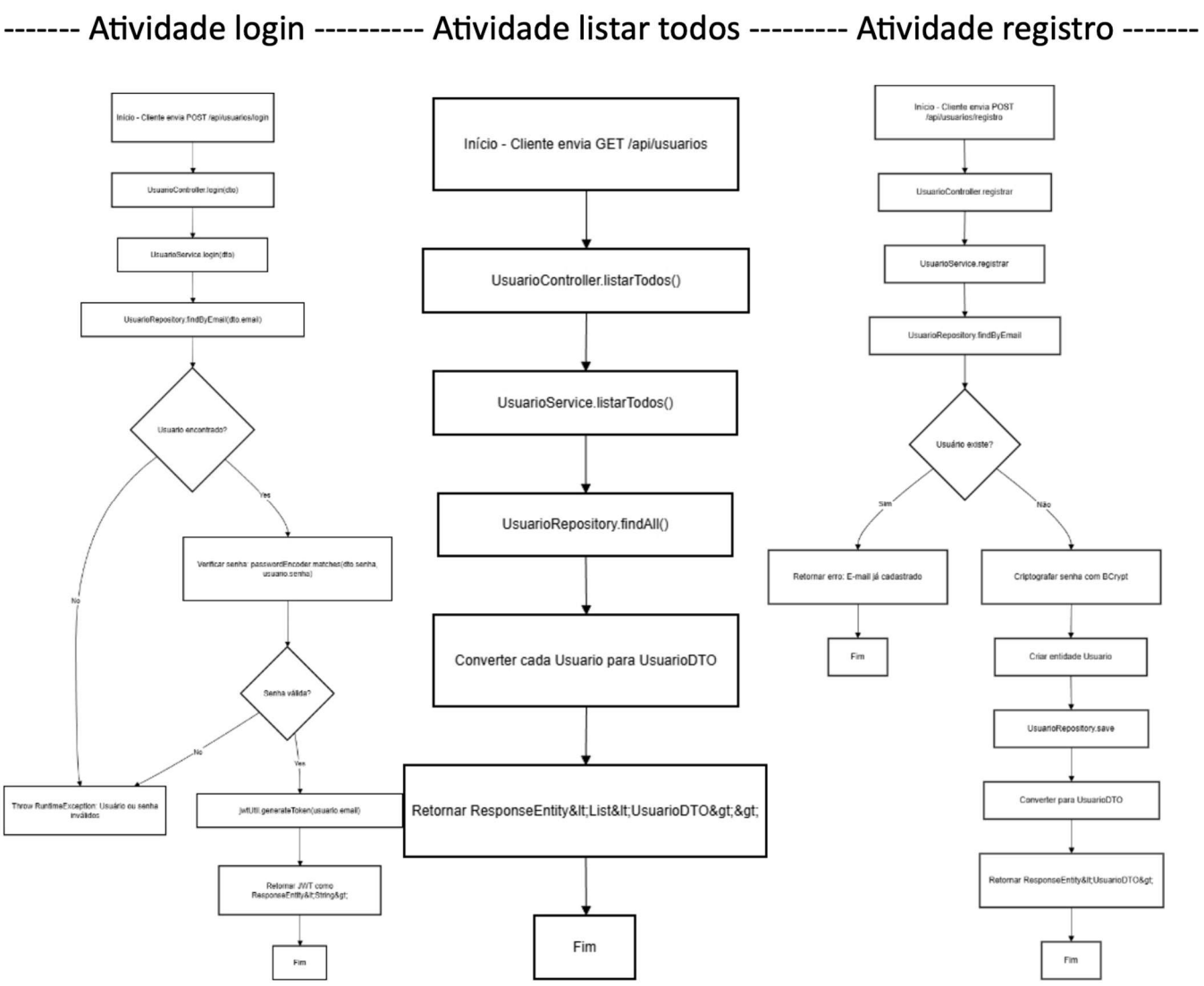
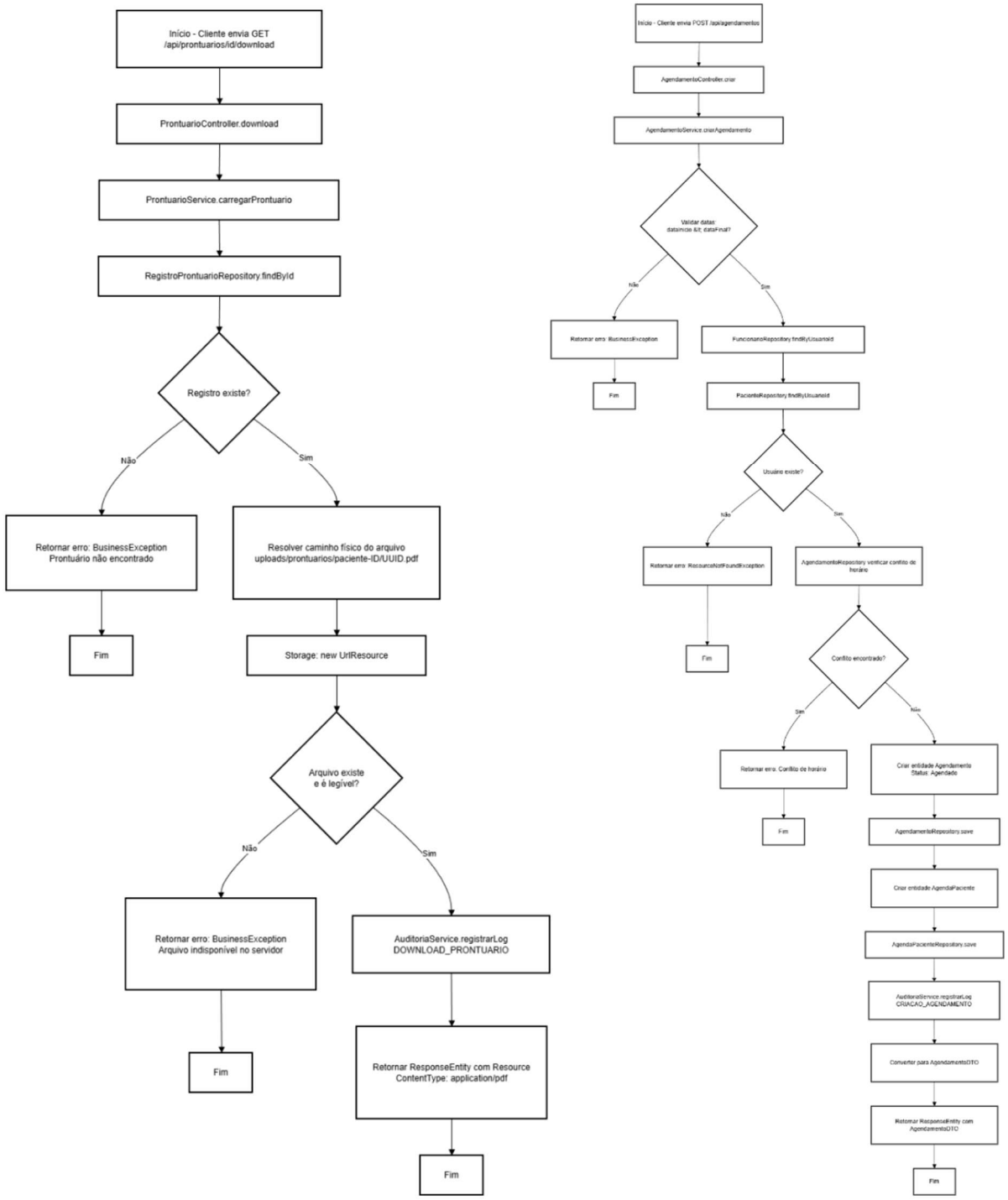


Figura 05 – Diagrama de atividade da aplicação

--------Atividade Download Prontuário----------Atividade Gestão Consulta----------



Fonte: Autoria Própria

4.5 Interfaces com o usuário

Ao abrir o sistema do AMDE Digital a tela inicial (figura 06) é a tela de login, os usuários inserem o seu login tanto como paciente, visitante, médico ou administrador, assim é

possível acessar o sistema e ingressar na plataforma na tela principal (figura 07), onde é possível acessar o dashboard com as principais funções do sistema, tendo um resumo das consultas realizadas, pacientes e informações gerais, facilitando aos utilitários visualizarem as principais informações de forma rápida e sucinta, temem possui o menu lateral com as funcionalidades do sistema.

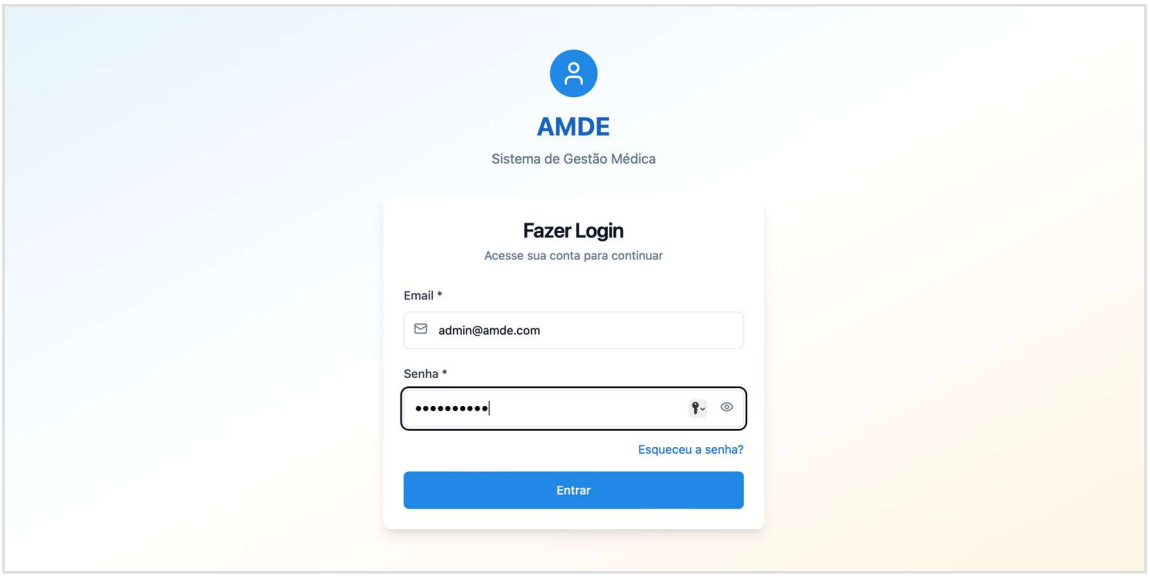


Figura 06 – Tela de Login AMDE Digital

Fonte: Autoria Própria

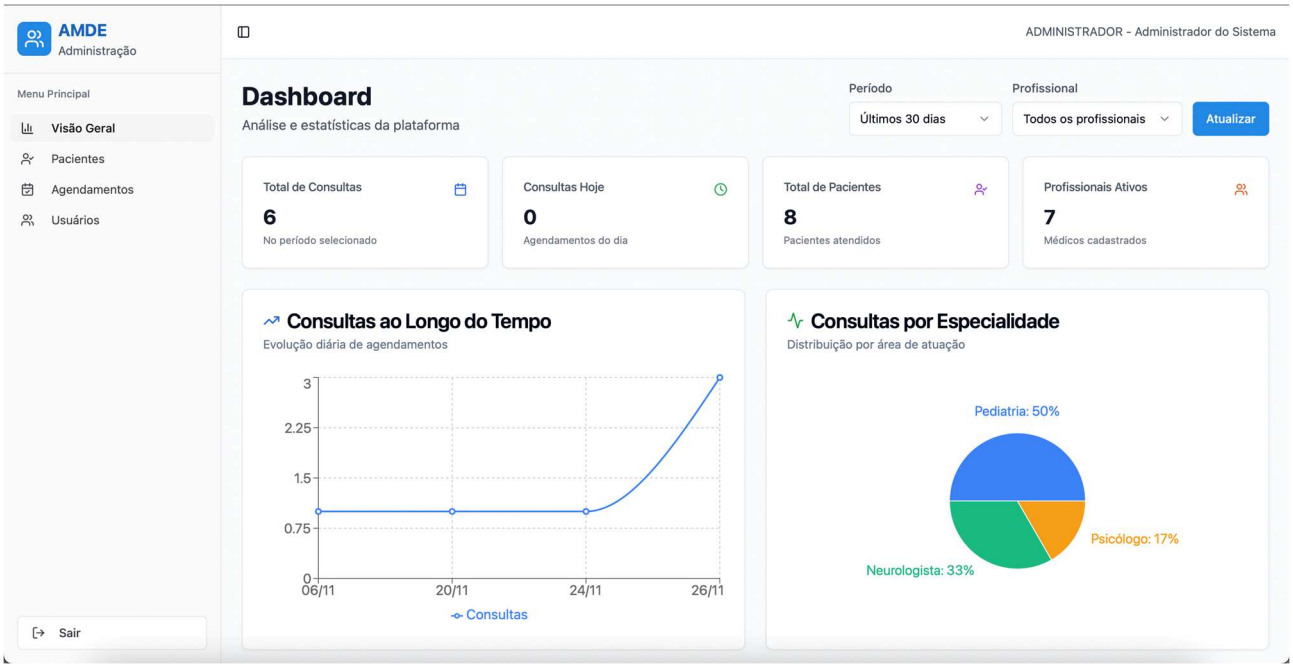


Figura 07 – Tela Inicial

Fonte: Autoria Própria

Na tela de pacientes (figura 08) é possível consultar os pacientes cadastrados e suas informações, tendo a possibilidade de buscar os pacientes diretamente pelo filtro para encontrar o desejado(a), selecionar o tipo de paciente, e escolher a ordem que serão mostrados os pacientes, podendo ser decrescente ou ascendente. Ao clicar no paciente ira exibir uma tela com o prontuário do paciente (figura 09), o administrador poderá adicionar informações ao prontuário do paciente, que é separado por categorias especificas solicitadas pela entidade para o controle de informações do paciente.

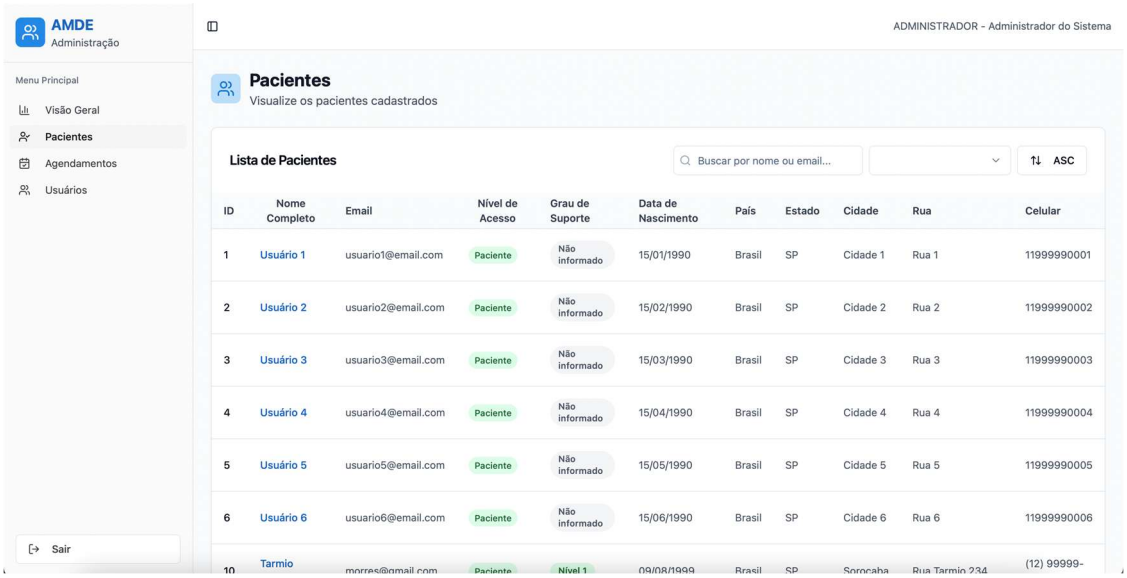


Figura 08 – Tela de Pacientes

Fonte: Autoria Própria

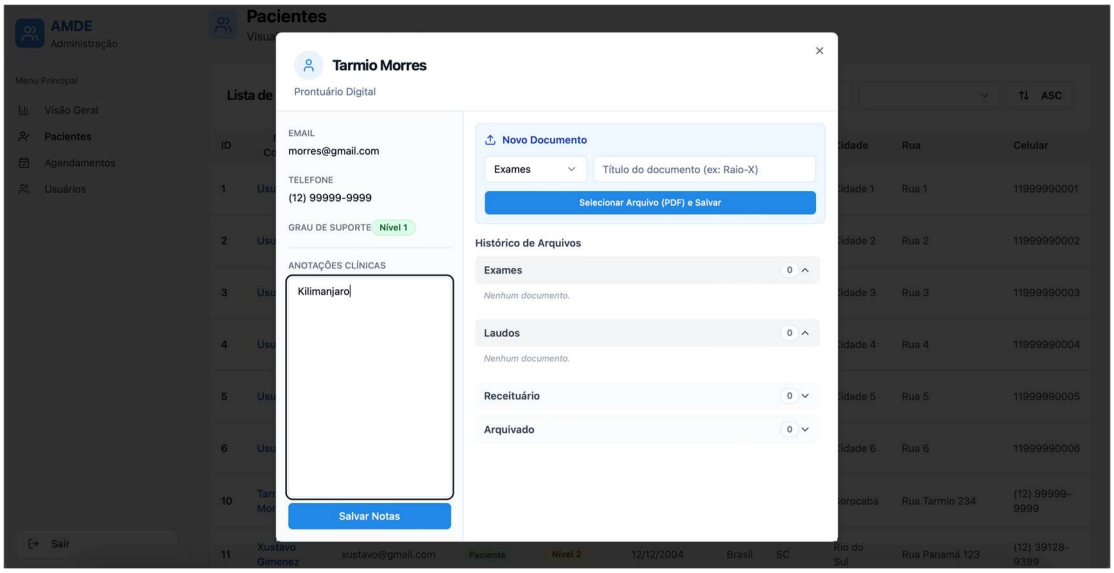


Figura 09 – Edição dos pacientes

Fonte: Autoria Própria

Voltando ao menu temos a aba de Usuários, entrando nesta tela (figura 10) primeiro vemos um resumo geral dos usuários, dividindo-os por categorias, também estarão as informações de login dos usuários, somente o administrador tem acesso, podendo editar, adicionar ou excluir usuários, para a visualização da lista de usuários é possível filtrar por usuários específicos, ordenar a visualização ou escolher o tipo de usuário (administrador, medico e paciente).

Por fim temos a tela de agendamentos (figura 11), que é o local onde ficam registradas as consultas da AMDE, também sendo o lugar onde é possível registra-las, edita-las ou exclui-las, na tabela são registradas as informações principais como horário, o medico que atendeu, a especialidade do mesmo, o paciente que foi atendido, o tipo de agendamento feito podendo ser consulta, exame, terapia, etc (esses tipos específicos ficaram a critério da instituição decidir) , e o status da consulta, para obter um controle de quantas foram realizadas ou canceladas.

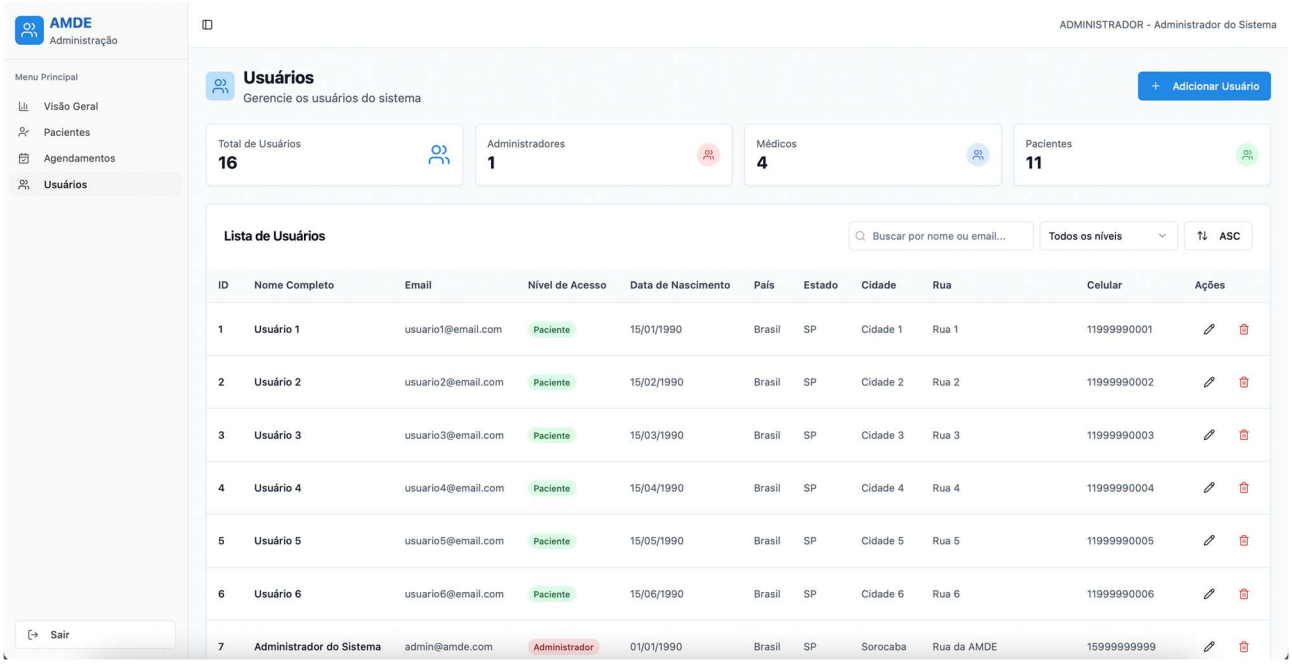


Figura 10 – Tela de Usuários

Fonte: Autoria Própria

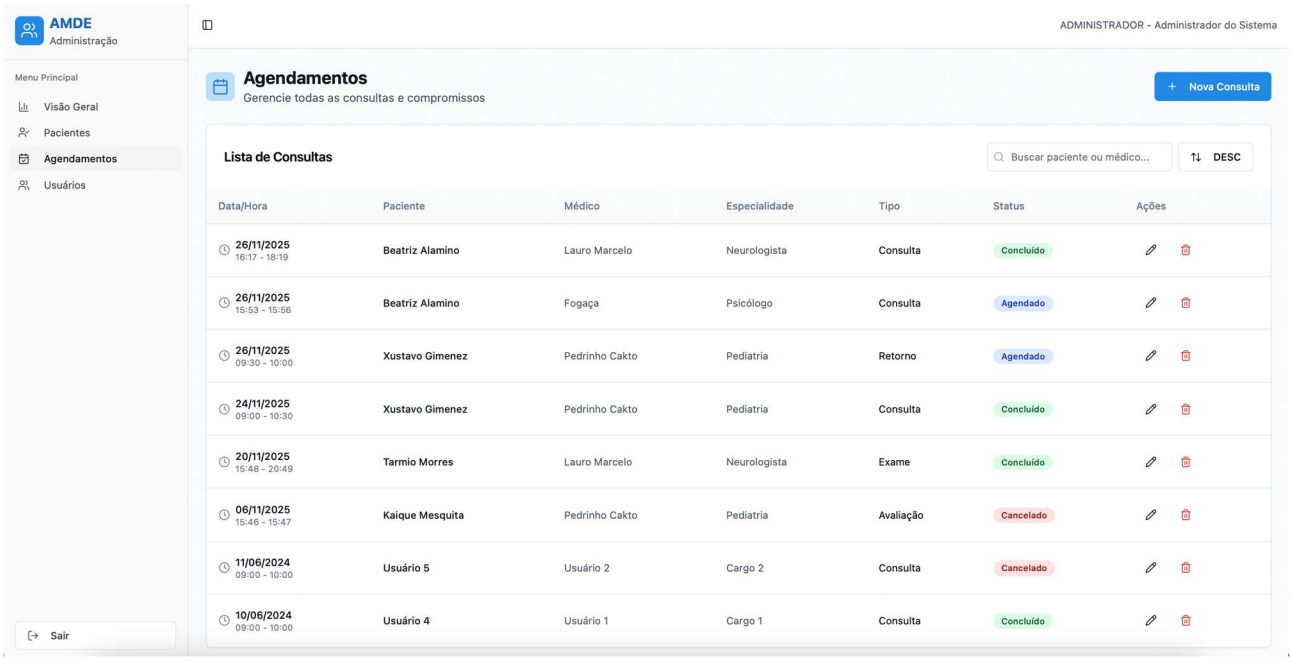


Figura 11 – Tela de Agendamentos

Fonte: Autoria Própria

5. Implementação

A implementação do sistema AMDE na instituição adota uma arquitetura em nuvem (*Cloud Computing*) robusta e escalável, hospedada na Amazon Web Services (AWS). A aplicação segue um modelo desacoplado, onde o Backend (API) é executado em containers Docker isolados para garantir segurança e estabilidade, enquanto o Frontend é distribuído via CDN (Rede de Distribuição de Conteúdo) para acesso rápido e alta disponibilidade. A integridade e persistência dos dados são asseguradas por um serviço de banco de dados relacional gerenciado, com rotinas automáticas de backup. Todo o processo de atualização e implantação é automatizado através de esteiras de CI/CD (Integração e Entrega Contínuas), garantindo que novas funcionalidades cheguem à instituição de forma ágil, segura e auditável.

5.1 Versionamento do Código – GitHub

O versionamento do código do projeto AMDE Digital foi realizado utilizando o GitHub como plataforma central de controle de versões. O repositório oficial contém todo o código-fonte do back-end, front-end e arquivos de configuração Docker.
As principais práticas adotadas foram:

- Uso da branch `main` para versões estáveis.
- Branch `develop` para funcionalidades em desenvolvimento.
- Branches específicas para novas features seguindo o padrão: **feature/nome-da-funcionalidade**
- Commits padronizados com mensagens claras e objetivas.
- Utilização de Pull Requests para revisão de código entre os membros do grupo.

Essa estrutura garante organização, rastreabilidade e colaboração eficiente entre os desenvolvedores.

O código fonte da aplicação esta disponível no seguinte repositório:
http://github.com/EnricoMDP/projeto-amde

O sistema já está disponível online na seguinte URL:
http://dtzu1m0v2dkwx.cloudfront.net/

5.2 Pipeline dos Projetos – GitHub Actions

Para automatizar testes, validações e geração de builds, foi configurado um pipeline no **GitHub Actions**, garantindo automação e padronização das entregas.

O pipeline executa as seguintes etapas:

- Checkout do código
- Build do backend (Java/Spring Boot)
- Build do frontend (React/Vite)
- Execução de testes
- Verificação de qualidade de código
- Geração da imagem Docker (quando necessário)
- Deploy automatizado em ambiente de produção

Essa automação reduz erros humanos e torna o processo de entrega mais confiável.

5.3 Estágios de Commit – GitHub

Os commits foram organizados em três estágios principais:

- **Commit Local:**

  Desenvolvedor cria alterações em sua máquina, garantindo que o código compila e funciona.

- **Commit na Branch de Feature:**

  O commit é enviado para o GitHub na branch correspondente, permitindo testes individuais e integração futura.

- **Merge com revisão (Pull Request):**

  Antes de integrar o código ao repositório principal (`develop` ou `main`), outro membro revisa as alterações.

Essa estratégia evita problemas em produção e mantém a qualidade do projeto.

5.4 Processo de Implantação e Entrega de Versão

O processo de implantação (deploy) do AMDE Digital segue estas etapas:

- Build e testes automatizados no pipeline
- Geração de imagem Docker da aplicação
- Envio da imagem para o Docker Hub (ou repositório privado)
- Substituição da versão anterior no servidor
- Reinicialização dos containers

- Validação do ambiente

O ambiente de produção utiliza containers Docker para garantir padronização e estabilidade entre máquinas diferentes.

5.5 Gerenciamento de Entrega Contínua (CI/CD)

Foi adotado o modelo de **Entrega Contínua (Continuous Delivery)**, no qual cada alteração aprovada nas branches principais gera automaticamente:

- Build validado
- Testes executados
- Atualização do ambiente de homologação
- Possibilidade de deploy com apenas um comando ou ação no GitHub

Benefícios alcançados:

- Menos erros em produção
- Mais rapidez nas entregas
- Transparência do processo
- Histórico completo de alterações

5.6 Guia de Instalação do Sistema AMDE

5.6.1 Softwares Complementares Essenciais (Pré-requisitos)

Quadro 11. Softwares necessários

Git	Gerenciamento de código-fonte.	Necessário para clonar o repositório do projeto no GitHub.
Docker Engine & Docker Compose	Virtualização de containers.	Essencial para rodar o PostgreSQL, Spring Boot, o Keycloak e o Kafka/RabbitMQ de forma isolada e padronizada.
Java Development Kit (JDK) 17+	Ambiente de execução e compilação do Back-end.	Requisito do Spring Boot.

Maven ou Gradle	Ferramenta de build do Back-end.	Usado para compilar e testar Java.
Node.js (LTS) & npm/Yarn	Ambiente de execução para o Front-end.	Necessário para rodar os projetos React/Angular (Web) e React Native.

Passo a Passo de Instalação -

1. Clonagem do Repositório (GitHub)

   Primeiro, obtenha o código-fonte do projeto. Na pasta de destino, execute:

   # Substitua <URL_DO_REPOSITORIO> pela URL real

   git clone <URL_DO_REPOSITORIO>

   cd <nome-do-repositorio>

2. Configuração do Docker e Variáveis de Ambiente

   O projeto deve conter um arquivo docker-compose.yml que orquestra todos os serviços (Banco de Dados, Keycloak, Mensageria).

   Verifique as Variáveis: Crie um arquivo `.env` na raiz do projeto ou verifique as variáveis de ambiente necessárias nos arquivos de configuração do Spring Boot (`application.yml` ou `application.properties`) e no `docker-compose.yml`.

   Exemplo de variáveis críticas:

   POSTGRES_USER=amdeuser

   POSTGRES_PASSWORD=amdepass

   KEYCLOAK_ADMIN_USER=admin

   KEYCLOAK_ADMIN_PASSWORD=senhaforte

   KAFKA_BROKER=kafka:9092

   Inicialize os Serviços de Infraestrutura: Inicie o banco de dados (PostgreSQL), o gerenciador de autenticação (Keycloak) e a mensageria (Kafka/RabbitMQ) via Docker Compose.

   docker-compose up -d postgres keycloak kafka # ou rabbitmq, dependendo da sua escolha

3. Build e Execução do Back-end

Passo 3.1: Build (Compilação)

Navegue até o diretório (ex: `cadastro-prontuarios-ms/`, `agendamento-ms/`) e execute o build.

Comando (Exemplo com Maven):

Na pasta de cada: ./mvnw clean package -DskipTests

Comando (Exemplo com Gradle):

Na pasta de cada: ./gradlew clean build -x test

Passo 3.2: Execução em Container

Após o build, o Docker Compose deve ser capaz de subir os containers.

Comando:

Volte para a raiz do projeto e execute: docker-compose up -d —build

4. Instalação e Execução do Front-end (Web)

1. Instalação de Dependências:

cd frontend-web # Ou o nome do diretório do projeto web (React/Angular)

npm install # ou yarn install

2. Execução:

npm start # ou yarn start / ng serve, dependendo do framework

5. Configuração do Keycloak

Após iniciar o Keycloak, acesse sua interface administrativa (Geralmente `http://localhost:8080/auth` ou porta configurada) e crie/configure:

- Realm: (Ex: amde-realm)
- Clientes (Clients): Para o front-end web.
- Usuários e Permissões (Roles): Configure os níveis de acesso (Administrador, Médicos, Assistentes, etc.), conforme documentado no `PROJETO INTEGRADOR I DOC COMPLETO.docx`.

5.7 Estimativa de Custos

A estimativa de custos para este projeto, voltado para uma instituição como a AMDE, será dominada por custos operacionais (OpEx), recomendamos o uso da AWS[15] para o sistema ficar online pela nuvem.

[15] Disponível em: https://aws.amazon.com/pt/

Tabela 01. Custos estimados

SERVIÇO	CUSTO MENSAL ESTIMADO ($ / R$)	CUSTO ANUAL ESTIMADO (R$)
AWS	$240,00 / R$114,00	$20,00 / R$1368,00

6. Conclusão

O desenvolvimento do AMDE Digital representou um marco significativo na busca pela modernização dos processos internos da Associação Amigos dos Deficientes (AMDE). A partir da constatação de que muitos procedimentos ainda eram executados de forma manual e fragmentada, este Projeto Integrador se propôs a analisar, compreender e solucionar as principais dificuldades enfrentadas pela instituição, especialmente no que diz respeito ao gerenciamento de pacientes, prontuários, documentos clínicos, consultas e controle de presença.

Ao longo deste trabalho, foi possível identificar a necessidade urgente de digitalização e automatização dos fluxos de informação, visando reduzir retrabalho, minimizar riscos de perda de dados sensíveis e aumentar a eficiência operacional. O levantamento de requisitos, realizado por meio de entrevistas, observações diretas e análise documental, forneceu uma base sólida para compreender as reais demandas dos profissionais da instituição. Esse processo possibilitou o alinhamento das funcionalidades do sistema às atividades essenciais desempenhadas pela AMDE, garantindo que a solução proposta fosse prática, aplicável e capaz de gerar impacto imediato no cotidiano da organização.

A modelagem do sistema, contribuiu para uma visão clara da organização interna do software. Além disso, o projeto colocou em prática conceitos essenciais de engenharia de software, como separação de camadas, modularidade, reuso de componentes, controle de versões e integração contínua (CI/CD). A implementação dos pipelines no GitHub Actions e o gerenciamento das branches no GitHub reforçaram o profissionalismo e a maturidade técnica alcançada ao longo do desenvolvimento.

A solução entregue contempla funcionalidades que atendem diretamente às necessidades identificadas: autenticação segura, gestão de usuários, controle de pacientes e prontuários, upload de documentos, organização de agendas e registro de presença. Além disso, o uso de padrões modernos de segurança, como tokens JWT e controle rigoroso de permissões, garante conformidade com a legislação vigente, especialmente no que diz respeito à LGPD.

Em síntese, o AMDE Digital cumpre plenamente seu objetivo principal: oferecer à instituição uma plataforma moderna, eficiente e segura para a gestão de prontuários e processos clínicos. O sistema representa não apenas uma evolução tecnológica, mas também uma oportunidade de transformação administrativa e organizacional para a AMDE. Espera-se que, com sua implementação e uso contínuo, a instituição consiga otimizar

recursos, melhorar a qualidade do atendimento e ampliar a confiabilidade das informações armazenadas.

Como perspectiva futura, o sistema pode ser expandido com novos módulos, como dashboards de indicadores clínicos, relatórios gerenciais, integração com sistemas externos, recursos de notificação automática, ou até mesmo a introdução de ferramentas baseadas em inteligência artificial para análise preditiva. Essas possíveis evoluções reforçam o caráter escalável e adaptável da solução, garantindo sua longevidade e aderência às necessidades crescentes da instituição.

Apresentamos o sistema finalizado para os administradores da AMDE, foram feitos testes pelos mesmo aprovando o sistema, mas foi decidido que o uso efetivo somente se iniciará no começo de 2026, para que todos as registros novos sejam feitos na aplicação. Assim, esperamos que o desenvolvimento do AMDE Digital não apenas responda ao problema inicialmente proposto, mas também abra caminhos para inovação contínua e aprimoramento dos serviços prestados pela Associação Amigos dos Deficientes, demonstrando o valor da tecnologia como agente transformador dentro do contexto da saúde e da gestão institucional.

Referências

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS (ABNT). *NBR 6023: Informação e documentação – Referências – Elaboração.* Rio de Janeiro: ABNT, 2018. (Norma que rege a formatação desta seção).

DOCKER. *Docker Documentation.* Docker, Inc., [entre o ano de fundação e o ano atual]. Disponível em: [https://docs.docker.com/]. Acesso em: **[06/09/2025]**.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley Professional, 2002.

GAMMA, Erich et al. *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos.* Tradução de: Design Patterns: Elements of Reusable Object-Oriented Software. Porto Alegre: Bookman, 2000.

HARDT, Dick. *The OAuth 2.0 Authorization Framework.* Internet Engineering Task Force (IETF), RFC 6749, October 2012. Disponível em: [https://datatracker.ietf.org/doc/html/rfc6749]. Acesso em: **[27/08.2025]**.

KEYCLOAK. *Keycloak Documentation.* Red Hat, [entre o ano de fundação e o ano atual]. Disponível em: [https://www.keycloak.org/documentation]. Acesso em: **[15/08/2025]**.

NEWMAN, Sam. Building Microservices: Designing Fine-Grained Systems. 1. ed. Sebastopol, CA: O'Reilly Media, 2015.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation.* [S.l.]: [s.n.], [entre o ano de fundação e o ano atual]. Disponível em: [https://www.postgresql.org/docs/]. Acesso em: **[24/05/2025]**.

SPRING. *Spring Boot Documentation.* Pivotal, [entre o ano de fundação e o ano atual]. Disponível em: [https://spring.io/projects/spring-boot]. Acesso em: **[25/05/2025]**.

TURNBULL, James. The Docker Book: Containerization is the New Virtualization. S.l.: The Docker Book, 2014.

Glossário

AMDE: (Associação Amigos dos Deficientes). Nome da instituição assistida e foco central do projeto integrador.

APIs: (Application Programming Interfaces). Interfaces de Programação de Aplicações. São conjuntos de regras e protocolos que permitem a comunicação e troca de dados entre diferentes softwares ou sistemas.

APIs REST: (Representational State Transfer). Arquitetura de APIs que utiliza os verbos HTTP (GET para consulta, POST para criação, PUT para atualização, DELETE para exclusão) para realizar operações de forma padronizada, eficiente e sem estado. É a arquitetura mais comum para a comunicação entre Front-end e Back-end.

CI/CD: (Continuous Integration / Continuous Delivery). Integração Contínua / Entrega Contínua. Prática de DevOps que automatiza o processo de compilação, teste e implantação de código, garantindo que as alterações sejam integradas e entregues no ambiente de produção de forma rápida e segura.

CRUD: (Create, Read, Update, Delete). Conjunto das quatro operações básicas e essenciais que um sistema deve ter sobre os dados persistidos: Criar novos registros, Ler/Consultar dados existentes, Atualizar informações e Excluir registros.

Framework: Estrutura de código reutilizável que fornece uma base para o desenvolvimento de software, definindo padrões, fornecendo bibliotecas e abstraindo complexidades, o que acelera e padroniza o desenvolvimento (ex: Spring Boot, React).

JWT: (JSON Web Token). Padrão aberto (RFC 7519) para a criação de tokens com informações codificadas e assinadas digitalmente.

OSC: Organizações da Sociedade Civil são entidades sem fins lucrativos que objetivam cooperar com o Estado no atendimento ao interesse público.

PEP: (Prontuário Eletrônico do Paciente). Versão digital e centralizada do histórico médico, dados de saúde e informações de atendimento do paciente, crucial para a gestão segura do sistema AMDE.

TICs: (Tecnologia da Informação e Comunicação). Conjunto de recursos tecnológicos (computadores, softwares, redes, internet) utilizados para criar, armazenar, processar e disseminar informações.